Two pointer

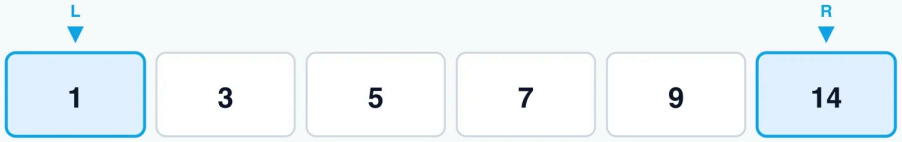
4 min read Updated 16 Jul 2026 Topic 60 of 77

23C. Two Pointer (10 problems)

Pattern recognition: "Sorted array", "Find pair/triplet", "Container/area problems", "Remove duplicates in-place"

Two pointers: sorting lets each step discard a whole row of comparisons

Find a pair summing to 12 in a sorted array



1 + 14 = 15 > 12 → too big, move R left

1 + 14 = 15

Too big? move R left (drop the largest). Too small? move L right (drop the smallest).

23.21 Two Sum (Sorted Array)

JAVASCRIPT Copy

```
function twoSum(arr, target) {
  let lo = 0, hi = arr.length - 1;
  while (lo < hi) {
    const sum = arr[lo] + arr[hi];
    if (sum === target) return [lo, hi];
    if (sum < target) lo++;
```

```
  else hi--;
 return [-1, -1];
}
```

23.22 Three Sum

JAVASCRIPT Copy

```
function threeSum(nums) {
  nums.sort((a, b) => a - b);
  const result = [];
  for (let i = 0; i < nums.length - 2; i++) {
    if (i > 0 && nums[i] === nums[i - 1]) continue;
    let lo = i + 1, hi = nums.length - 1;
    while (lo < hi) {
      const sum = nums[i] + nums[lo] + nums[hi];
      if (sum === 0) {
        result.push([nums[i], nums[lo], nums[hi]]);
        while (lo < hi && nums[lo] === nums[lo + 1]) lo++;
        while (lo < hi && nums[hi] === nums[hi - 1]) hi--;
        lo++; hi--;
      } else if (sum < 0) lo++;
      else hi--;
    }
  }
  return result;
}
```

Time: O(n²). **Space:** O(1) excluding output.

23.23 Container With Most Water

JAVASCRIPT Copy

```
  while (lo < hi) {
    const area = Math.min(heights[lo], heights[hi]) * (hi - lo);
    max = Math.max(max, area);
    if (heights[lo] < heights[hi]) lo++;
    else hi--;
  }
  return max;
}
```

23.24 Remove Duplicates from Sorted Array

JAVASCRIPT Copy

```
function removeDuplicates(nums) {
  if (nums.length === 0) return 0;
  let slow = 0;
  for (let fast = 1; fast < nums.length; fast++) {
    if (nums[fast] !== nums[slow]) { slow++; nums[slow] = nums[fast]; }
  }
  return slow + 1;
}
```

23.25 Trapping Rain Water

JAVASCRIPT Copy

```
function trap(height) {
  let lo = 0, hi = height.length - 1;
  let leftMax = 0, rightMax = 0, water = 0;
  while (lo < hi) {
    if (height[lo] < height[hi]) {
      leftMax = Math.max(leftMax, height[lo]);
      water += leftMax - height[lo];
```

```
      water += rightMax - height[hi];
      hi--;
    }
  }
  return water;
}
```

Time: O(n). **Space:** O(1).

23.26 Valid Palindrome

Approach: Strip non-alphanumerics and lowercase, then converge two pointers from both ends comparing characters.

JAVASCRIPT Copy

```
function isPalindrome(s) {
  const clean = s.toLowerCase().replace(/[^a-z0-9]/g, '');
  let left = 0, right = clean.length - 1;
  while (left < right) {
    if (clean[left] !== clean[right]) return false;
    left++;
    right--;
  }
  return true;
}
```

Two pointers from both ends, skip non-alphanumeric, compare case-insensitive.

23.27 Merge Two Sorted Arrays

Approach: Walk one pointer per array, always taking the smaller front element, then drain whichever array remains.

```javascript
function mergeSortedArrays(a, b) {
  const result = [];
  let i = 0, j = 0;
  while (i < a.length && j < b.length) {
    if (a[i] ≤ b[j]) result.push(a[i++]);
    else result.push(b[j++]);
  }
  while (i < a.length) result.push(a[i++]);
  while (j < b.length) result.push(b[j++]);
  return result;
}
```

Two pointers, one per array, compare and push smaller.

23.28 Partition Array

Approach: Advance a left pointer past elements below the pivot and a right pointer past elements at or above it, swapping the out-of-place pair when both stall.

JAVASCRIPT Copy

```javascript
function partition(nums, pivot) {
  let left = 0, right = nums.length - 1;
  while (left ≤ right) {
    if (nums[left] < pivot) {
      left++;
    } else if (nums[right] ≥ pivot) {
      right--;
    } else {
      [nums[left], nums[right]] = [nums[right], nums[left]];
      left++;
      right--;
    }
  }
  return nums;
}
```

23.29 Dutch National Flag (Sort Colors)

Approach: Three pointers (low, mid, high) sweep once: send 0s to the front, 2s to the back, and leave 1s in the middle.

JAVASCRIPT Copy

```
function sortColors(nums) {
  let low = 0, mid = 0, high = nums.length - 1;
  while (mid ≤ high) {
    if (nums[mid] === 0) {
      [nums[low], nums[mid]] = [nums[mid], nums[low]];
      low++;
      mid++;
    } else if (nums[mid] === 1) {
      mid++;
    } else {
      [nums[mid], nums[high]] = [nums[high], nums[mid]];
      high--;
    }
  }
  return nums;
}
```

Three pointers (lo, mid, hi). 0s go to lo, 2s go to hi, 1s stay in middle. Single pass O(n).

23.30 Pair with Given Difference

Approach: Sort the array, then slide two pointers to grow or shrink the difference until it equals the target (or the pointers cross).

JAVASCRIPT Copy

```
  const d = Math.abs(diff);
  let i = 0, j = 1;
  while (j < arr.length) {
    const cur = arr[j] - arr[i];
    if (i !== j && cur === d) return true;
    if (cur < d) j++;
    else i++;
    if (i === j) j++;
  }
  return false;
}
```

Sort + two pointers. If `arr[j] - arr[i] < target` advance j, if greater advance i.

QUICK CHECK — no marks, just for you.

1. What does this snippet print?

JAVASCRIPT Copy

```
function twoSum(arr, target) {
  let lo = 0, hi = arr.length - 1;
  while (lo < hi) {
    const sum = arr[lo] + arr[hi];
    if (sum === target) return [lo, hi];
    if (sum < target) lo++;
    else hi--;
  }
  return [-1, -1];
}

console.log(JSON.stringify(twoSum([1, 3, 4, 6, 8, 11], 10)));
console.log(JSON.stringify(twoSum([1, 3, 4, 6, 8, 11], 20)));
```

- ○ `[2,3]`
  `[-1,-1]`

- [1,4]
  [-1,-1]
- [4,6]
  [-1,-1]

Check answer

2. What does this snippet print?

JAVASCRIPT Copy

```javascript
function removeDuplicates(nums) {
  if (nums.length === 0) return 0;
  let slow = 0;
  for (let fast = 1; fast < nums.length; fast++) {
    if (nums[fast] !== nums[slow]) { slow++; nums[slow] = nums[fast]; }
  }
  return slow + 1;
}

const nums = [1, 1, 2, 2, 3];
const k = removeDuplicates(nums);
console.log(k);
console.log(JSON.stringify(nums));
```

- 5
  [1,2,3,2,3]
- 3
  [1,2,3]
- 3
  [1,2,3,2,3]
- 3
  [1,1,2,2,3]

3. According to the lesson, how does the Dutch National Flag (Sort Colors) approach sort an array of 0s, 1s, and 2s?

- ○ It repeatedly swaps adjacent out-of-order elements until the array is sorted, costing O(n log n).
- ○ It counts how many 0s, 1s, and 2s exist in one pass, then rewrites the array in a second pass.
- ○ It uses three pointers (lo, mid, hi), pushing 0s toward lo and 2s toward hi in a single O(n) pass.
- ○ It uses two pointers and requires the array to be sorted before the scan begins.

Check answer

‹ Previous
Searching

Next ›
Sliding window

LEARN MORE

ARTICLES & DOCS

Two-pointers technique (GfG)
geeksforgeeks.org

VIDEOS



Two pointers in 7 minutes
YouTube



Master two pointers & fast/slow

Frontend System Desi...



Visual intro to two pointers

YouTube

Hand-picked references for this lesson.